

CLASE N°02

FACULTAD DE INGENIERÍA
EN SISTEMAS, ELECTRONICA
E INDUSTRIAL

fisei

MANEJO DE PARÁMETROS, PASO DE VALORES Y CLASE **CALCULADORA**

PRACTICANDO LA MODULARIDAD Y EL COMPORTAMIENTO DE OBJETOS EN JAVA.

¿Qué se busca aprender?

- Modularidad: organizar el código en clases y métodos.
- Objetos: representar elementos del mundo real mediante atributos y comportamientos.
- Métodos: definir las acciones que puede realizar cada objeto.
- Reutilización: aprovechar el código existente sin tener que escribirlo nuevamente.



PARÁMETROS VS. ARGUMENTOS

OBJETIVO: Explicar la diferencia conceptual y práctica entre ambos términos.

DEFINICION:

- Parámetro: Es la variable declarada en la firma del método. Funciona como un espacio reservado para recibir datos.
- Argumento: Es el valor real que le enviamos al método cuando lo llamamos desde otra parte del código.

EJEMPLO CALCULADORA:

```java
// 'a' y 'b' son PARÁMETROS (Definición del método)
public double sumar(double a, double b) {
    return a + b;
}

// '15.5' y '4.5' son ARGUMENTOS (Llamada al método)
double resultado = miCalc.sumar(15.5, 4.5);
```

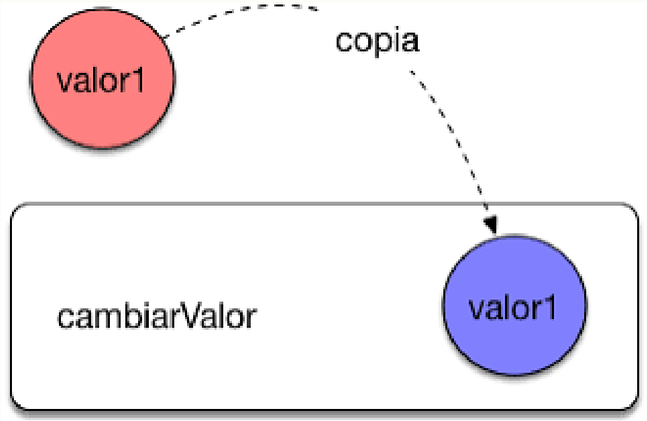
EXPLICACION:

- public double: Obliga al método a devolver un resultado con decimales.
- double a, double b (Parámetros): Casilleros vacíos que esperan los datos que van a llegar.
- return a + b;: Hace la operación y saca el resultado hacia afuera del método.
- 15.5 y 4.5 (Argumentos): Los valores reales y concretos que le inyectamos al llamar al método.
- double resultado = ...: La variable que captura y guarda el valor devuelto para poder usarlo.

PASO POR VALOR EN JAVA

- Puntos clave:
- En Java, los argumentos SIEMPRE se pasan por valor.
- Cuando le envías una variable a un método, Java entrega una copia de ese valor.
- Modificar la variable del parámetro dentro del método NO afecta la variable original que estaba afuera.

- PREGUNTA:
- “Si declaro int x = 5; y se lo paso a un método que le suma +10 a su parámetro, ¿cuánto vale x fuera del método después de ejecutarlo?”



MÚLTIPLES PARÁMETROS

- Un método puede recibir la cantidad de parámetros que necesite.
- Se separan por comas (,).
- Pueden ser del mismo tipo de dato o de tipos combinados (String, int, double, boolean).

EJEMPLO DE CODIGO:

```java
public void registrarOperacion(String tipoOperacion, double resultado, boolean exitosa) {
    System.out.println("Operación: " + tipoOperacion);
    System.out.println("Resultado: " + resultado);
    System.out.println("¿Exitosa?: " + exitosa);
}
```

- EXPLICACION:

- Un método puede recibir la cantidad de parámetros que necesite.
- Se separan por comas (,).
- Pueden ser del mismo tipo de dato o de tipos combinados (String, int, double, boolean).
- Uso interno: El método recibe esos tres valores y los muestra organizados en la consola.

CLASE CALCULADORA

- Objetivo: Mostrar la estructura de la clase práctica que van a codificar en vivo en el IDE.
- Código a presentar:

```java
public class Calculadora {

    // Método void sin retorno
    public void mostrarMenu() {
        System.out.println("=== CALCULADORA BÁSICA EN JAVA ===");
        System.out.println("1. Sumar | 2. Restar | 3. Multiplicar | 4. Dividir");
    }

    // Métodos con retorno y parámetros
    public double sumar(double a, double b) {
        return a + b;
    }

    public double restar(double a, double b) {
        return a - b;
    }

    public double multiplicar(double a, double b) {
        return a * b;
    }

    // Validación especial: División entre cero
    public double dividir(double a, double b) {
        if (b == 0) {
            System.out.println("Error: No se puede dividir entre cero.");
            return 0;
        }
        return a / b;
    }
}
```

- EXPLICACION
- public class Calculadora: El molde principal que agrupa todas las acciones (métodos) de nuestra calculadora.
- public void mostrarMenu(): Ejecuta una acción visual (imprimir en consola) sin devolver ningún valor.
- public double sumar / restar / multiplicar(...): Métodos que reciben dos parámetros, hacen el cálculo y devuelven el resultado con return.
- if (b == 0) (Validación): Controla un error lógico antes de operar; evita que el programa falle al intentar dividir entre cero.
- return a / b;: Realiza y devuelve la división normal cuando el denominador es válido (b es diferente de cero).

EJECUCIÓN EN VIVO (CLASE MAIN)

- Contenido: Cómo instanciar la clase Calculadora y probar cada uno de sus métodos en la clase principal.
- Código a presentar:

```java
public class Main {
    public static void main(String[] args) {
        // Instanciación del objeto
        Calculadora calc = new Calculadora();

        // Llamada al método void
        calc.mostrarMenu();

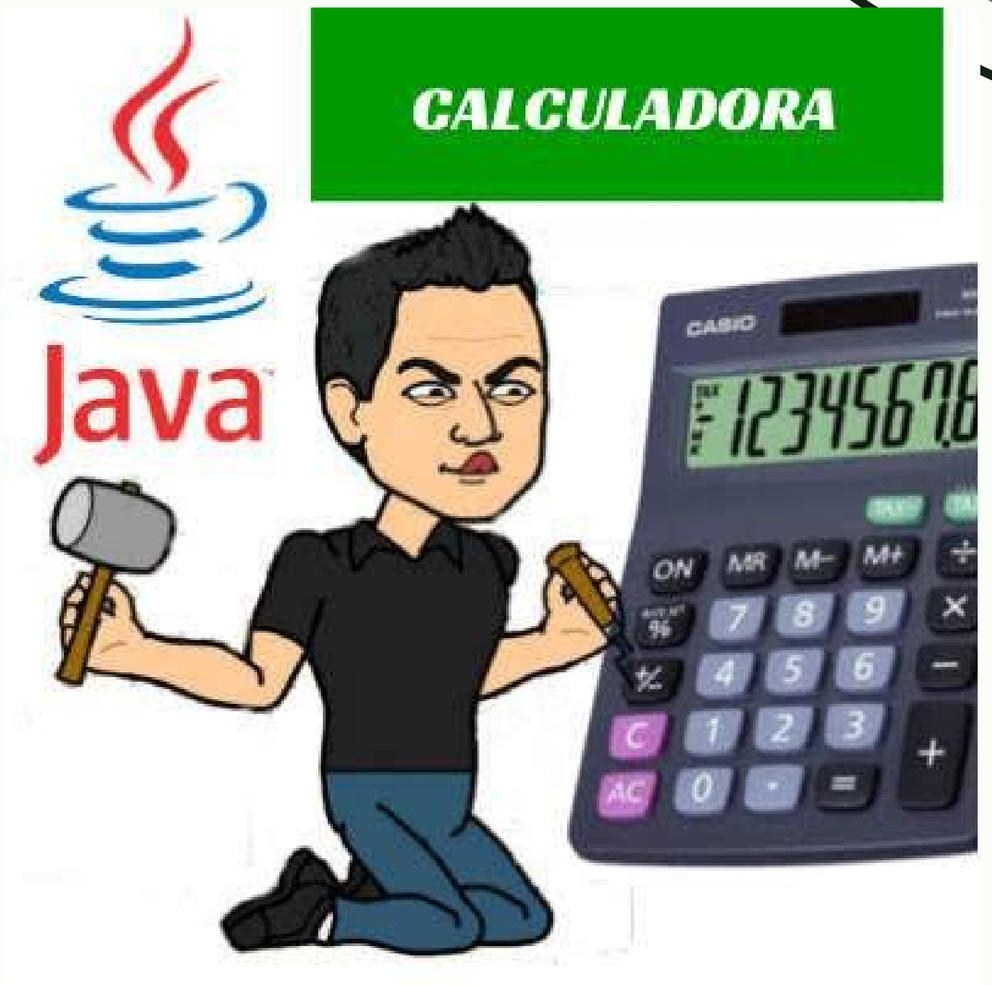
        // Invocación de métodos con retorno
        double totalSuma = calc.sumar(10.5, 5.5);
        System.out.println("Resultado Suma: " + totalSuma);

        double totalDiv = calc.dividir(20, 0); // Prueba de la validación
    }
}
```

- public class Main y main(...): El punto de partida obligatorio donde Java comienza a ejecutar el programa.
- Calculadora calc = new Calculadora();: Instancia el objeto, creando un ejemplar real de la clase para poder usar sus métodos
- calc.mostrarMenu();: Llama y activa el método void para desplegar las opciones en pantalla.
- double totalSuma = calc.sumar(10.5, 5.5);: Envía los argumentos 10.5 y 5.5, ejecuta la operación y captura el resulta devuelto en la variable.
- calc.dividir(20, 0);: Prueba en vivo la validación del método ante una división entre cero.

DIFERENCIAS DE LOS EJERCICIOS:

- Clase Calculadora: Muestra el molde o la fábrica de los métodos. Aquí es donde se programan las reglas, los cálculos, los parámetros y las validaciones (como la de la división por cero). Por sí sola, esta clase solo define cómo se hacen las cosas, pero no las ejecuta.
- Clase Main: Muestra la ejecución o el uso en la vida real. Aquí es donde se crea el objeto (new Calculadora()) y se invoca o llama a cada método pasándole argumentos reales (10.5, 5.5, etc.) para ver los resultados en la consola.



CONCLUSIONES DE LA SESIÓN



Flexibilidad:

Flexibilidad: Los parámetros permiten que los métodos procesen distintos datos sin reescribir código.



Seguridad

El paso por valor protege las variables originales de ser modificadas accidentalmente.



Validación

Los métodos deben ser capaces de validar sus entradas (como prevenir divisiones para b == 0).



Preparación

Listos para la sesiónde "Completar Código" con la clase Producto


Gracias por su atencion